

PPMT Trie Architecture

Deep Analysis Report

4-Level Trie (N1, N2, N3, N4) Architecture Audit
6 Fatal Failures Identified in Production
Data Strategy + File Modification Map

Version 0.40.34 | Code Audit Date: 2026-06-20

Repository: github.com/coverdraft/ppmt

Table of Contents

1. Executive Summary
2. FALLO 1: N1 y N2 son una mentira - Pools compartidos vacios
3. FALLO 2: Alphabet Size (alpha) identico en los 4 niveles
4. FALLO 3: SAX solo mira precio, ignora volumen
5. FALLO 4: Sin contexto BTC
6. FALLO 5: Nodos solo miran 1 paso adelante
7. FALLO 6: Umbrales de regimen inadecuados para timeframes bajos
8. Verificacion de Base de Datos
9. Mapa de Archivos a Modificar
10. Estrategia de Data
11. Plan de Implementacion por Fases
12. Preparacion para Terminal (Fase 5)

1. Executive Summary

Este documento presenta el resultado de una auditoria profunda del codigo fuente de PPMT v0.40.34, enfocada en la arquitectura de 4 niveles del Trie (N1, N2, N3, N4). La auditoria confirma que el motor base (SAX, Trie, Prediccion) es funcionalmente correcto, pero la infraestructura que lo rodea tiene 6 fallos fatales que impiden que el sistema opere cualquier token en timeframes bajos (1m, 5m, 15m). Estos fallos no son bugs de logica interna sino de disenio arquitectonico: las piezas existen en el codigo pero estan desconectadas o configuradas incorrectamente.

El hallazgo mas critico es que N1 (Universal) y N2 (Asset Class) son una mentira en produccion: cada token construye su propio Trie aislado. Los pools compartidos (`__UNIVERSAL__`, `__CLASS_*`) estan vacios en la base de datos. Aunque la infraestructura de pools existe en `storage.py` y el metodo `attach_storage()` existe en `ppmt.py`, los pools nunca se han llenado porque nunca se ha ejecutado un build secuencial multi-token con storage adjunto. Esto significa que un token nuevo no tiene de donde sacar experiencia: su N1 y N2 estan vacios y el sistema no puede hacer Transfer Learning.

El segundo hallazgo critico es que el Alphabet Size (alpha) es el mismo para los 4 niveles, lo que hace que N1 (que deberia ser una red de seguridad universal con pocos patrones muy repetidos) tenga miles de combinaciones posibles que nunca se llenan con datos insuficientes. Con $\alpha=8$ y `pattern_length=5`, N1 tiene $8^5 = 32,768$ combinaciones posibles. Con solo 1000 velas de datos, la inmensa mayoria de patrones en N1 estan vacios o tienen 1 sola observacion (sin significado estadistico).

La solucion requiere implementar las 5 fases descritas en la directriz tecnica, empezando por la Fase 1 (alpha diferenciado, pools compartidos, SAX dual) que es la base sin la cual nada funciona. Este documento detalla cada fallo con evidencia del codigo fuente, identifica los archivos exactos a modificar, y define la estrategia de datos necesaria para poblar correctamente los tries compartidos.

2. FALLO 1: N1 y N2 son una mentira - Pools compartidos vacios

2.1 Evidencia del Codigo

La arquitectura de 4 niveles esta disenada para que N1 sea compartido entre TODOS los tokens (pool universal) y N2 sea compartido entre tokens de la misma clase de activo (pool de clase). El codigo en `storage.py` define las claves especiales `__UNIVERSAL__` y `__CLASS_*<asset_class>` para estos pools. El metodo `attach_storage()` en `ppmt.py` (linea 195) permite que durante el build, cada observacion se acumule en buffers de N1 y N2 que se flushean a los pools compartidos al final del build.

Sin embargo, la verificacion de la base de datos muestra que los pools compartidos estan completamente vacios:

```
SELECT symbol, level FROM tries WHERE symbol LIKE '__%'
-- Resultado: 0 filas. NO existen __UNIVERSAL__ ni __CLASS_* en la DB.
```

Mientras tanto, cada token tiene sus propias entradas N1, N2, N3 en la tabla tries, con tamanos casi identicos (diferencia menor al 0.01% en serializacion JSON), confirmando que N1=N2=N3 para cada token individual. Esto se debe a que el codigo entra en el camino de retrocompatibilidad cuando no hay storage adjunto (lineas 479-492 de ppmt.py):

```
if self._storage is not None:
    # FIX-1B: accumulate in buffers (fast)
    self._n1_buffer.insert_with_observations(...)
    self._n2_buffer.insert_with_observations(...)
    self.trie_n3.insert_with_observations(...)
else:
    # Backwards-compat (no storage): N1/N2/N3 all receive locally
    for trie in [self.trie_n1, self.trie_n2, self.trie_n3]:
        trie.insert_with_observations(...)
```

2.2 Por que los Pools estan Vacios

Aunque `attach_storage()` se llama correctamente en `paper_trader.py` (linea 918), `realtime.py` (linea 760) y `terminal/server.py` (lineas 809, 1792, 2933), el problema es que nunca se ha ejecutado un build secuencial multi-token. La base de datos solo contiene 3 tokens (BTC/USDT, ETH/USDT, SOL/USDT) con apenas 1000 velas cada uno, y cada uno se construyo de forma aislada. Cuando se hace build de un solo token, el buffer de N1 y N2 se llena con los patrones de ese token y se guarda en el pool compartido. Pero despues de un build de un solo token, los pools `__UNIVERSAL__` y `__CLASS_blue_chip` deberian existir. La razon por la que no existen es que los builds previos se hicieron ANTES de que se implementara FIX-1B (v0.40.3), o se hicieron sin adjuntar storage.

2.3 Consecuencia para Tokens Nuevos

Cuando un token nuevo (ej. PEPE/USDT) llega al sistema, el flujo en `paper_trader.py` (lineas 906-927) intenta cargar los tries asi: `all_tries = storage.load_all_tries(symbol, asset_class)`. Este metodo busca N1 en `__UNIVERSAL__` y N2 en `__CLASS_meme`. Como no existen, devuelve None para ambos niveles. El codigo de fallback (linea 923) hace `trie_n1 = all_tries['n1'] or engine.trie_n1`, que usa el `trie_n1` vacio del engine recién creado. Resultado: N1 y N2 estan vacios, y la confidence ponderada depende unicamente de N3 (que tambien esta vacio para un token nuevo) y N4 (que esta vacio y mal calibrado). El motor no genera senales y el sistema esta muerto para ese token.

2.4 Solucion Requerida

La solucion requiere tres pasos: (1) Ejecutar un build secuencial multi-token con storage adjunto, donde BTC se construye primero para estabilizar N1, luego ETH, luego el resto por clase. (2) Verificar que los pools `__UNIVERSAL__` y `__CLASS_*` existen en la DB con miles de patrones combinados. (3) Eliminar el fallback `or engine.trie_n1` que oculta que los pools estan vacios. Si N1 compartido no existe, el sistema debe saberlo y actuar en consecuencia (usar solo N3 con sizing reducido), no fingir que N1 existe usando una copia local.

3. FALLO 2: Alphabet Size (alpha) identico en los 4 niveles

3.1 Estado Actual

Actualmente, el SAXEncoder se crea una unica vez con un alphabet_size fijo, y ese mismo encoder se usa para construir los 4 niveles del Trie. En el constructor de PPMT (ppmt.py linea 107-155), se pasa un unico sax_alphabet_size que se usa para el SAXEncoder compartido. Los 4 tries (N1, N2, N3, N4) reciben exactamente los mismos simbolos SAX y los mismos patrones. No hay ningun mecanismo para usar diferentes alpha por nivel.

El archivo profiles.py (lineas 71-79) define TIMEFRAME_ALPHA_DEFAULTS que asigna alpha segun el timeframe (1m: alpha=4, 5m: alpha=4, 15m: alpha=4, 1h: alpha=3), pero este alpha es para TODOS los niveles, no por nivel individual. El CalibrationEngine puede descubrir el mejor alpha para un token, pero siempre aplica ese alpha uniformemente a N1, N2, N3 y N4.

3.2 Impacto Matematico

Con alpha=5 y pattern_length=5, el numero de patrones posibles es $5^5 = 3,125$. Con alpha=8 (valor por defecto en config/default.yaml), son $8^5 = 32,768$ patrones posibles. Si N1 (Universal) usa alpha=5 y solo tiene datos de 3 tokens con 1000 velas cada uno (unos 3,000 simbolos SAX en total), genera unos 2,995 patrones de longitud 5. Esto significa que en promedio cada patron se observa menos de 1 vez. La inmensa mayoria de los 3,125 patrones posibles estan vacios o tienen una sola observacion, lo que hace que la confidence sea inestable y no significativa.

Si N1 usara alpha=3, tendria $3^5 = 243$ patrones posibles. Con los mismos 3,000 simbolos SAX, cada patron se observaria en promedio 12.3 veces. Esto da a cada nodo metadata estadisticamente significativa (win_rate, expected_move, confidence estables). Un token nuevo que llegue al sistema con alpha=3 en N1 tiene 243 'combinaciones maestras' donde buscar, y es muy probable que su patron actual encuentre un match en N1. Con alpha=5 o alpha=8, es muy probable que no encuentre nada.

3.3 Comparacion de Alpha por Nivel

Nivel	Alpha Actual	Patrones Posibles	Alpha Propuesto	Patrones Propuestos	Razon
N1 (Universal)	4-8	1,024 - 32,768	3	243	Red de seguridad, se llena rapido
N2 (Asset Class)	4-8	1,024 - 32,768	3-4	243 - 1,024	Memes: alpha=3, BlueChip: alpha=4
N3 (Per-Token)	4-8	1,024 - 32,768	4-5	1,024 - 3,125	Tiene data especifica, mas granularidad
N4 (Per-Token+Region)	4-8	1,024 - 32,768	4-5	1,024 - 3,125	Igual que N3

Tabla 1: Comparacion de Alpha actual vs propuesto por nivel de Trie. Los patrones posibles se calculan como α^5 (pattern_length=5).

3.4 Solucion Requerida

Se necesitan multiples SAXEncoders, uno por nivel. Cada encoder tiene su propio alphabet_size. Durante el build, cada observacion se codifica con el encoder correspondiente al nivel antes de insertarse en el trie de ese nivel. Durante el match, el patron actual se codifica con cada encoder y se busca en el trie correspondiente. Esto requiere modificar: (1) PPMT.__init__() para crear 4 encoders, (2) PPMT.build() para codificar con el encoder de cada nivel, (3) PPMT.match() para codificar y buscar con el encoder de cada nivel, (4) StreamingPatternBuffer para manejar multiples encoders en tiempo real, y (5) FuzzyMatcher para soportar comparacion entre simbolos de diferentes alfabetos.

4. FALLO 3: SAX solo mira precio, ignora volumen

4.1 Estado Actual del SAX

El SAXEncoder actual (sax.py) usa una estrategia 'ohlcv' que crea un composite aditivo de tres features: body_position (peso 0.4), direction (peso 0.35), y vol_signal (peso 0.25). Sin embargo, el vol_signal es solo un ratio relativo del volumen actual vs la media movil de 20 periodos, normalizado al rango [0, 1]. Este volumen relativo se funde con los otros dos features en un unico valor composite que luego se discretiza en un solo simbolo SAX. El problema fundamental es que toda la informacion de volumen se comprime en una dimension que compite con la informacion de precio por la misma banda de variabilidad.

En la practica, cuando el precio se mueve fuertemente (direction cercano a +1 o -1), el body_position y el direction dominan el composite, y el vol_signal tiene poco impacto en el simbolo final. Una vela que sube 2% con volumen 10x se codifica casi igual que una vela que sube 2% con volumen normal, porque el vol_signal (normalizado a [0,1]) solo contribuye 0.25 al composite, mientras que direction contribuye hasta 0.35. Para memes y tokens nuevos, el volumen ES la senal principal: un pump de volumen sin movimiento de precio es a menudo el precursor de un movimiento grande, y el sistema actual no puede distinguirlo.

4.2 Solucion: SAX Dual

La solucion propuesta es un SAX Dual donde el precio y el volumen se codifican de forma independiente con sus propios encoders SAX (cada uno con su alpha). El patron final que se inserta en el Trie es una concatenacion o tupla de ambos simbolos. Por ejemplo, con alpha_precio=3 y alpha_volumen=3, cada posicion del patron seria una tupla (precio_sym, vol_sym) como ('a', 'x'), lo que da $3 \times 3 = 9$ simbolos compuestos por posicion, y $9^5 = 59,049$ patrones posibles. Esto es demasiado, asi que se recomienda usar alpha_volumen=2 para N1 (alto/bajo volumen), lo que da $3 \times 2 = 6$ simbolos compuestos y $6^5 = 7,776$ patrones en N1. Para N3 (per-token), se puede usar alpha_precio=4 y alpha_volumen=3, dando 12 simbolos compuestos y $12^5 = 248,832$ patrones (factible con suficientes datos).

La implementacion requiere: (1) Crear un SAXDualEncoder que mantenga dos SAXEncoders internos, (2) Modificar los metodos encode/encode_incremental para producir tuplas de simbolos, (3) Modificar PPMTTrie para aceptar simbolos compuestos (o tuplas) como claves, (4) Modificar FuzzyMatcher para calcular distancias entre tuplas, y (5) Actualizar todos los puntos de serializacion/deserializacion en storage.py.

5. FALLO 4: Sin contexto BTC

El motor PPMT no tiene ningun mecanismo para considerar el estado del mercado general (representado por BTC) al tomar decisiones de trading en otros tokens. Cuando evalua PEPE/USDT, no sabe si BTC esta en tendencia alcista, bajista, o lateral. Esto es fatal en crypto, donde BTC dicta la direccion del mercado: una senal LONG en DOGE mientras BTC cae un 3% tiene una probabilidad de exito significativamente menor que la misma senal cuando BTC sube.

Actualmente, el unico contexto que tiene el motor es el regimen del propio token (N4), que como vimos en el Fallo 6, esta mal calibrado para timeframes bajos. No existe ninguna variable, parametro, o filtro que represente el estado de BTC/USDT en el pipeline de decision. La clase RealtimeTrader procesa cada token de forma completamente independiente, sin compartir informacion entre ellos.

La solucion es un filtro post-prediccion que no toca el Trie: el sistema obtiene el regimen actual de BTC/USDT (usando RegimeDetector o incluso una senal SAX simple de BTC) y aplica un multiplicador a la confidence de la senal del altcoin. Si la senal es LONG y BTC esta en `trending_down`, se multiplica la confidence por 0.3 (o se rechaza directamente). Si BTC esta en `volatile_extremo`, se rechaza cualquier senal en altcoins/memes. Este filtro se implementa en el gestor de posiciones, despues de la prediccion pero antes de la ejecucion. No requiere cambios al Trie ni al SAX.

6. FALLO 5: Nodos solo miran 1 paso adelante

La metadata de cada nodo del Trie (`BlockLifecycleMetadata` en `metadata.py`) almacena un campo `continuation_nodes` que es una lista de simbolos que siguieron historicamente a ese patron. Sin embargo, esta lista solo registra el SIMBOLO INMEDIATAMENTE SIGUIENTE (`next_symbol`), no la secuencia completa. Cuando el motor predice el futuro, solo sabe que despues del patron actual, el simbolo mas probable es 'f'. Pero no sabe que viene despues de 'f': podria ser 'g' (continuacion alcista) o 'x' (reversa). El motor no se entera de la divergencia hasta que el precio toca el Stop Loss fisico.

La `PredictionEngine` (`prediction.py`) intenta compensar esto caminando el Trie hacia adelante (`_walk_path`), pero este camino depende de que los nodos intermedios existan en el Trie del token especifico. Si el patron observado diverge del esperado en el segundo paso, el motor no tiene un mecanismo explicito para detectar esa divergencia y cerrar la posicion. La senal de salida depende del `FuzzyMatcher.check_continuation()` que compara el ultimo simbolo observado con los simbolos esperados, pero no compara secuencias completas.

La solucion tiene dos partes: (1) En la metadata del nodo, reemplazar la lista de simbolos individuales por un diccionario de secuencias esperadas con sus frecuencias. Ejemplo: `{('f','g','h'): 35, ('f','x','y'): 12}`. Esto le dice al motor no solo que viene despues, sino las proximas 3 velas SAX esperadas. (2) En el monitor de posiciones abiertas, en cada nueva vela, comparar la secuencia SAX real con la secuencia esperada. Si divergen por encima de un umbral (ej. 2 de 3 simbolos distintos), ejecutar cierre por `reason='pattern_broken'`, sin esperar al SL/TP.

7. FALLO 6: Umbrales de regimen inadecuados para timeframes bajos

Los umbrales de regimen en `RegimeThresholds` (`thresholds.py`) usan `simple_vol_cutoff=0.08` (8%) y `simple_move_cutoff=0.02` (2%). Estos umbrales fueron diseñados para timeframes de 1h o superiores, donde un movimiento del 2% en una hora es significativo. Sin embargo, en timeframes de 1m, 5m y 15m, estos umbrales son absurdamente altos: el 92% de las velas caen en 'ranging' porque los movimientos intra-vela en 1m rara vez superan el 2%.

El metodo `detect_simple()` en `regime.py` usa estos umbrales para clasificar el regimen durante el build del Trie (linea 152-198). Cada observacion que se inserta en N4 (`RegimePartitionedTrie`) se etiqueta con un regimen, y esa etiqueta determina en cual de los 4 sub-tries (`trending_up`, `trending_down`, `ranging`, `volatile`) se inserta. Si el 92% de las observaciones van a 'ranging', entonces el sub-trie de ranging se vuelve identico a N3, y los otros 3 sub-tries estan casi vacios. N4 no aporta informacion adicional sobre el regimen.

7.1 Umbrales Propuestos por Timeframe

Timeframe	move_cutoff	vol_cutoff	Justificacion
1m	0.008 (0.8%)	0.025 (2.5%)	Movimiento tipico BTC 1m: 0.1-0.5%
5m	0.012 (1.2%)	0.035 (3.5%)	Movimiento tipico BTC 5m: 0.3-0.8%
15m	0.018 (1.8%)	0.050 (5.0%)	Movimiento tipico BTC 15m: 0.5-1.5%
1h (actual)	0.020 (2.0%)	0.080 (8.0%)	Valor actual, solo valido para 1h+

Tabla 2: Umbrales de regimen propuestos por timeframe. Los valores para 1m/5m/15m estan calibrados para que la distribucion de regimenes sea mas equilibrada (aprox. 30% trending, 40% ranging, 20% volatile, 10% quiet).

La implementacion requiere que `RegimeThresholds` acepte el timeframe como parametro y ajuste los umbrales automaticamente. El metodo `detect_simple()` ya recibe un `DataFrame` que sabe su timeframe, asi que solo necesita leer los umbrales correctos de `RegimeThresholds`.

8. Verificacion de Base de Datos

La base de datos SQLite en `~/.ppmt/ppmt.db` (6.0 MB) contiene las tablas: `assets`, `ohlcv`, `tries`, `engine_states`, `signals`, `trades`, `validations`. La verificacion revela:

Aspecto	Estado	Detalle
Pools compartidos	VACIOS	No existen <code>__UNIVERSAL__</code> ni <code>__CLASS_*</code> en tabla <code>tries</code>
N1=N2=N3 por token	CONFIRMADO	BTC n1=384KB, n2=385KB, n3=385KB (identico)
N4 diferente	CONFIRMADO	BTC n4=419KB (<code>RegimePartitionedTrie</code> , mas grande)
Tokens en DB	3 tokens	BTC/USDT, ETH/USDT, SOL/USDT
OHLCV por token	1000 rows	Solo 1000 velas por timeframe (insuficiente)

Aspecto	Estado	Detalle
Timeframes	1m, 5m, 15m	BTC tiene 3 TFs, ETH/SOL solo 5m y 15m
Rango temporal	~10 dias	BTC 15m: 9-19 Jun 2026 (solo 10 dias)
Tabla assets	VACIA	0 assets registrados (aunque hay datos en ohlcv)

Tabla 3: Estado de la base de datos PPMT. Los datos son insuficientes para poblar los tries compartidos y para obtener predicciones estadisticamente significativas.

El dato mas critico es que solo hay 1000 velas por timeframe. Con window_size=7 y alpha=4 para 1m, esto produce unos 142 simbolos SAX y 137 patrones de longitud 5. Esto es insuficiente para llenar ni siquiera el 5% de los 1,024 patrones posibles con alpha=4. Se necesitan al menos 10,000 velas por timeframe para empezar a tener metadata significativa, y idealmente 100,000+ para N1/N2 compartidos.

9. Mapa de Archivos a Modificar

A continuacion se detallan los archivos especificos que necesitan modificacion para cada tarea, con la linea de accion concreta:

Archivo	Tarea	Modificacion
src/ppmt/core/sax.py	1.1 + 1.3	Crear SAXDualEncoder con 2 SAXEncoders internos. Encode produce tuplas (precio_sym
src/ppmt/engine/ppmt.py	1.1 + 1.2	+PPMT.__init__: crear 4 SAXEncoders con alpha diferenciado. PPMT.build(): codificar
src/ppmt/core/trie.py	1.3 + 2.1	PPMTTrie: soportar simbolos compuestos (tuplas) como claves de children. TrieNode:
src/ppmt/core/metadata.py	2.1 + 2.4	BlockLifecycleMetadata: agregar expected_sequences (dict de secuencias->frecuencia
src/ppmt/core/matcher.py	2.3 + 2.2	FuzzyMatcher: soportar distancia entre simbolos compuestos. Agregar sequence_diver
src/ppmt/engine/weights.py	3.1	AdaptiveWeights: soportar redistribucion de pesos cuando N3 tiene < 20 patrones. A
src/ppmt/engine/prediction.py	2.1	PredictionEngine.predict(): usar expected_sequences del nodo para construir camino
src/ppmt/engine/signal.py	2.2	SignalGenerator: agregar logica de pattern_broken cuando la secuencia SAX real div
src/ppmt/engine/realtime.py	2.3	RealtimeTrader: agregar filtro BTC context post-prediccion. Hacer attach_storage o
src/ppmt/engine/paper_trader.py	2.3	PaperTrader: agregar filtro BTC context. Eliminar fallback or engine.trie_n1. Hacer
src/ppmt/core/regime.py	3.2	RegimeDetector.detect_simple(): aceptar timeframe como parametro para seleccionar
src/ppmt/core/thresholds.py	3.2	RegimeThresholds: agregar umbrales por timeframe (1m, 5m, 15m). Metodo for_timefram
src/ppmt/data/storage.py	3.3	Serializacion: soportar simbolos compuestos en JSON. Actualizar save_trie/load_trie
src/ppmt/data/classifier.py	3.3	AssetClassifier: agregar regla de new_launch (<24h de vida) y meme (<7 dias + volu
src/ppmt/engine/buffer.py	1.1	StreamingPatternBuffer: soportar multiples encoders SAX para tiempo real.

Tabla 4: Mapa completo de archivos a modificar por tarea. Los archivos estan ordenados por prioridad de modificacion (core primero, engine despues, data al final).

10. Estrategia de Data

10.1 Tokens Representativos

Para poblar correctamente los tries compartidos (N1 y N2), necesitamos datos de al menos 10-15 tokens representativos de todas las clases de activos. El objetivo es que N1 (Universal) capture patrones que son invariantes entre tokens (ej. 'sube-plano-subesube-baja' suele preceder una continuacion alcista sin importar el token), y que N2 (Asset Class) capture patrones especificos de cada clase (ej. los memes hacen pumps mas violentos pero mas cortos que los blue chips).

Clase	Cantidad	Tokens Sugeridos	Razon
Blue Chip	2	BTC/USDT, ETH/USDT	Estabilidad, mayor data disponible, N1 base
Large Cap	3	SOL/USDT, BNB/USDT, XRP/USDT	Liquidez alta, patrones estables
Mid Cap	3	LINK/USDT, AVAX/USDT, DOT/USDT	Diversidad sectorial
Meme	3	DOGE/USDT, PEPE/USDT, WIF/USDT	Alta volatilidad, volumen es clave
New Launch	2	2 tokens nuevos (<7 dias)	Validar safe defaults y transfer learning

Tabla 5: Tokens representativos sugeridos para el pipeline de data. Se necesitan al menos 13 tokens para cubrir todas las clases de activos.

10.2 Volumen de Datos por Timeframe

La cantidad de datos necesaria depende del timeframe y del alpha del nivel. Con $\alpha=3$ y $\text{pattern_length}=5$, N1 necesita que cada uno de los 243 patrones se observe al menos 10 veces para tener metadata significativa ($\text{confidence} > 0.3$). Esto requiere aproximadamente 2,430 observaciones de patrones, que a su vez necesitan unos $2,430 + 4 = 2,434$ simbolos SAX. Con $\text{window_size}=7$, esto requiere $2,434 \times 7 = 17,038$ velas como minimo para N1 solo. Pero como N1 es compartido entre todos los tokens, la data se acumula: si tenemos 13 tokens, cada uno necesita contribuir solo $17,038 / 13 = 1,311$ velas a N1. En la practica, queremos exceso para robustez, asi que apuntamos a 50,000+ velas por token para los blue chips.

Timeframe	W (Window)	Alpha N1	Alpha N3	90 dias (velas)	Patrones N1	Patrones N3
1m	7	3	4	129,600	~18,514	~18,514
5m	7	3	4	25,920	~3,703	~3,703
15m	5	3	4	8,640	~1,728	~1,728

Tabla 6: Volumen de datos estimado para 90 dias por timeframe. Patrones = velas / window - 4. Con 13 tokens x 90 dias, N1 acumularia ~240,000 patrones en 1m, garantizando que cada uno de los 243 patrones posibles se observe cientos de veces.

10.3 Estrategia de Descarga

La descarga debe ser paginada y respetar los rate limits de los exchanges. Se recomienda usar MEXC como fuente principal (el proyecto ya tiene `websocket_feed.py` con soporte MEXC), con Binance como respaldo. El orden de descarga es critico: primero BTC (maxima data, estabiliza N1), luego ETH, luego el resto por clase de activo (blue_chip primero, large_cap despues, etc.). Cada token se descarga en los 3 timeframes (1m, 5m, 15m) para los ultimos 90 dias. El volumen debe incluirse siempre (es requisito para SAX Dual).

Despues de la descarga, el build secuencial es obligatorio: se construyen los tries en el mismo orden que la descarga, con storage adjunto. Al final de cada build, se verifica que los pools `__UNIVERSAL__` y `__CLASS__` se han actualizado. Al finalizar todos los builds, se ejecuta una verificacion final: los pools deben tener miles de patrones combinados, y cada patron debe tener un `historical_count > 10` para ser estadisticamente significativo.

10.4 Validacion OOS

Despues de completar la Fase 1 y 2, se debe correr un replay OOS (ultimos 7 dias) con un token nuevo (ej. WIF o PEPE) que NO este en el set de build. Si N3 de PEPE esta vacio pero N1/N2 compartidos generan senales con `confidence > 0.5`, el sistema funciona: demuestra que el Transfer Learning esta operativo. La metrica clave no es la ganancia (es papel), sino que el motor genera senales con confidence significativa usando exclusivamente N1 y N2 cuando N3 esta vacio.

11. Plan de Implementacion por Fases

El plan sigue la directriz tecnica proporcionada, con 5 fases secuenciales. Cada fase depende de la anterior y no se debe avanzar sin completar y validar la fase actual.

FASE 1: Fundamentos

Tarea 1.1: Alpha Diferenciado por Nivel

Crear 4 SAXEncoders con alpha diferente. N1=3, N2=3-4, N3=4-5, N4=4-5. Modificar `PPMT.__init__`, `build()`, `match()`, `buffer.py`, `matcher.py`.

Tarea 1.2: Activar Pools Compartidos (FIX-1B)

Hacer `attach_storage()` obligatorio. Eliminar `fallback` or `engine.trie_n1`. Ejecutar build secuencial multi-token. Verificar pools en DB.

Tarea 1.3: SAX Dual (Precio + Volumen)

Crear `SAXDualEncoder`. Modificar `trie.py` para simbolos compuestos. Actualizar serializacion. Modificar fuzzy matcher.

FASE 2: Inteligencia de Decision

Tarea 2.1: Secuencias Forward

Agregar `expected_sequences` a `BlockLifecycleMetadata`. Modificar `build()` para registrar secuencias de 3 pasos. Modificar `PredictionEngine`.

Tarea 2.2: Pattern Divergence Exit

Agregar monitor en gestor de posiciones. Comparar SAX real vs esperado en cada vela. Cerrar por `pattern_broken` si `diverge > umbral`.

Tarea 2.3: Filtro Context BTC

Obtener regimen de BTC/USDT. Aplicar multiplicador a confidence de altcoins. LONG + BTC_down = conf * 0.3. BTC volatile = rechazar.

Tarea 2.4: Decaimiento Temporal

Agregar last_seen_timestamp a metadata. Aplicar decaimiento exponencial: $\text{conf} *= \exp(-\lambda * \text{dias_desde_ultimo})$.

FASE 3: Robustez para Cualquier Token

Tarea 3.1: Safe Defaults para Tokens Nuevos

Si N3 tiene < 20 patrones, redirigir peso a N1/N2. Reducir sizing proporcionalmente a inmadurez.

Tarea 3.2: Ajustar Umbrales de Regimen por Timeframe

RegimeThresholds.for_timeframe('1m') -> move=0.008, vol=0.025. Parametrizar detect_simple().

Tarea 3.3: Clasificador Asset Class a Prueba de Balas

<24h -> new_launch. <7dias + vol>20x -> meme. Resto por Market Cap.

FASE 4: Pipeline de Data

Tarea 4.1: Descarga Paginada Masiva

Script para 90 dias de 1m/5m/15m de 13+ tokens. Incluir volumen. Rate limiting.

Tarea 4.2: Build Secuencial Inteligente

Orden: BTC, ETH, resto por clase. Verificar pools despues de cada build.

FASE 5: Preparacion para Terminal

Tarea 5.1: Exponer Trie Maturity Score

API endpoint: /api/trie-maturity/{symbol} -> N1/N2/N3/N4 pattern counts y fill ratios.

Tarea 5.2: Exponer Decision Trace

API endpoint: /api/decision-trace/{symbol} -> razon completa de la decision.

Tarea 5.3: Exponer Pattern Matched

API endpoint: /api/pattern/{symbol} -> patron SAX, nivel, confidence, expected sequence.

Tarea 5.4: Exponer Expected vs Real Sequence

WebSocket event: sequence_update -> {expected: [...], real: [...], divergence: 0.3}.

12. Preparacion para Terminal (Fase 5)

La Fase 5 no construye la terminal, pero todo lo que se implemente en las fases 1-4 debe exponer los datos necesarios para cuando se construya. El sistema ya tiene un servidor FastAPI

(terminal/server.py) y un frontend HTML (terminal/static/index.html) que pueden extenderse. Los datos criticos a exponer son los siguientes cuatro indicadores que permiten al operador entender el estado del sistema y la razon de cada decision:

Dato	Descripcion	Endpoint/Evento	Fuente
Trie Maturity Score	Que tan lleno esta N1, N2, N3, N4	GET /api/trie_maturity	PPMT (N1, N2, N3, N4)
Decision Trace	Por que se tomo la decision. Ej: GONG/8PP/decision-0-66P/8PPDLE/1tBFCREJ1tCFCED(StCttrending_down)	GET /api/decision-trace	PPMT
Pattern Matched	Que patron SAX matcheo, de que nivel	GET /api/pattern_matched	PPMT
Expected Sequence	Si hay posicion abierta, mostrar WebSockets: es para que se puedan ver las SAX incremental	WebSockets	PPMT

Tabla 7: Datos a exponer para la terminal. Los endpoints son sugerencias; la implementacion final puede usar WebSocket exclusivamente para datos en vivo.

Este documento constituye la base analitica para la reconstruccion de la infraestructura PPMT. Las fases deben implementarse en orden estricto: sin la Fase 1 (alpha diferenciado + pools compartidos + SAX dual), las fases posteriores no pueden funcionar. La prioridad absoluta es el alpha por nivel: sin esto, N1 nunca sera una red de seguridad universal efectiva para tokens nuevos.